

RAPPORT DE STAGE

Développement de CyberDiag

Outil de diagnostic et d'audit cyber (OSINT, DNS, Nmap, VirusTotal)

Informations	Détails
Stagiaire	Manai Ousama
Entreprise d'accueil	CYBERSES
Tuteur entreprise	Romain Cerruti
Tuteur pédagogique	Emanuelle Seirin
Période de stage	12 janvier 2026 - 20 février 2026
Année scolaire	2025-2026

Sommaire

Remerciements	3
Introduction	4
I. Présentation de la mission	5
A. Contexte professionnel de la mission	5
1. Présentation de l'entreprise Cyberses	5
2. Interlocuteurs et documents de référence	6
3. Objectifs de la mission	6
4. Enjeux pour l'entreprise	7
B. Problèmes à résoudre, contraintes et limites	8
1. Problèmes identifiés	8
2. Contraintes de la mission	8
3. Limites identifiées.	9
II. Démarche suivie pour effectuer la mission	9
A. Processus suivi et méthode retenue	9
1. Méthode de travail	9
2. Présentation du processus adopté	10
B. Organisation des missions	11
1. Suivi et communication	11
2. Ressources mobilisées	11
III. Réalisation de la mission	12
A. Justification de la solution retenue	12
1. Comparatif des solutions existantes	12
2. Choix technologiques retenus	13
B. Étapes essentielles de la réalisation	14
1. Architecture modulaire de CyberDiag	14
2. Analyse des fonctionnalités développées	15
3. Difficulté principale rencontrée et résolution	17
IV. Évaluation des réalisations et des compétences mobilisées	18
A. Adéquation du travail	18
B. Compétences mises en œuvre	19
Conclusion	21
Glossaire	22
Bibliographie et webographie	23
Annexes	24
Annexe 1 - Extrait du fichier main.py (orchestrateur CyberDiag)	24
Annexe 2 - Exemple de rapport PDF généré par CyberDiag	26
Annexe 3 - Schéma de l'architecture modulaire	27
Annexe 4 - Capture de l'interface web Flask	28
Annexe 5 - Exemple de fichier de sortie JSON	29

Remerciements

Je tiens à remercier chaleureusement l'ensemble de l'équipe de Cyberses pour leur accueil et la confiance qui m'a été accordée tout au long de ce stage. Je remercie tout particulièrement Romain Cerruti, mon tuteur en entreprise, pour sa disponibilité, la clarté de ses orientations techniques et son accompagnement constant qui m'ont permis de progresser efficacement tout au long de la mission.

Je remercie également [NOM DU TUTEUR ÉCOLE], mon tuteur pédagogique à l'EPSI Montpellier, pour le suivi réalisé et les conseils apportés dans l'élaboration de ce rapport.

Enfin, je remercie l'ensemble des collaborateurs de Cyberses pour leur disponibilité et leur partage de connaissances dans les domaines de la cybersécurité et de l'OSINT, qui ont grandement contribué à la réussite de cette mission.

Introduction

Ce rapport de stage présente les travaux réalisés au sein de l'entreprise Cyberses dans le cadre de ma première année de BTS SIO option SISR (Solutions d'Infrastructure, Systèmes et Réseaux) à l'EPIS Montpellier. D'une durée de six semaines, du 12 janvier au 20 février 2026, ce stage m'a permis d'appréhender concrètement le monde professionnel de la cybersécurité offensive et de l'audit de sécurité.

La mission principale qui m'a été confiée par Romain Cerruti est le développement de CyberDiag, un outil d'analyse et de diagnostic cyber centré sur l'OSINT (Open Source INTelligence). Cet outil permet d'effectuer, à partir d'un nom de domaine, une analyse complète et automatisée de l'exposition d'une organisation sur Internet : découverte des enregistrements DNS, réputation du domaine via VirusTotal, collecte des adresses e-mail exposées via Hunter.io et scraping web, scan des ports ouverts via Nmap, enrichissement des données grâce à l'API Pappers pour les entreprises françaises, et génération de rapports PDF et JSON consolidés.

Ce document s'articule autour de quatre grandes parties : la présentation de la mission et de son contexte professionnel, la démarche méthodologique suivie, la réalisation technique détaillée de l'outil, et enfin l'évaluation critique des résultats obtenus et des compétences mobilisées.

CyberDiag est un outil d'audit OSINT développé en Python, à destination des consultants en cybersécurité de Cyberses. Il automatise la phase de reconnaissance préalable aux missions de pentest et d'audit.

I. Présentation de la mission

A – Contexte professionnel de la mission

1. Présentation de l'entreprise Cyberses

Cyberses est une entreprise spécialisée dans la cybersécurité et l'audit de sécurité informatique. Elle accompagne ses clients dans l'identification et la réduction de leur exposition aux risques numériques, en proposant des prestations d'audit, de tests d'intrusion (pentest), de veille sur les menaces et de conseil en sécurité. Ses clients sont des organisations de tailles variées – PME, ETI, collectivités – souhaitant évaluer et renforcer leur posture de sécurité face aux menaces extérieures.

Au sein de l'équipe technique de Cyberses, j'ai été intégré sous la supervision directe de Romain Cerruti, responsable technique et tuteur de stage. Ce contexte d'entreprise spécialisée m'a offert une immersion directe dans les pratiques réelles de l'audit de cybersécurité, avec accès à des méthodologies professionnelles et à des outils du secteur.

Critère	Détail
Secteur	Cybersécurité - Audit & Conseil
Spécialité	Pentest, OSINT, Veille cyber, Conseil en sécurité
Cibles clients	TPE/PME, ETI, Collectivités françaises
Localisation	France
Taille	Petite structure - équipe resserrée de consultants

2. Interlocuteurs et documents de référence

Le principal interlocuteur de la mission a été Romain Cerruti, qui a défini les besoins fonctionnels de l'outil, validé les choix techniques et orienté les priorités de développement à travers des points de suivi réguliers. Des échanges ponctuels avec d'autres membres de l'équipe ont permis d'affiner certaines fonctionnalités.

Les ressources documentaires mobilisées comprenaient : la documentation officielle des API utilisées (Hunter.io, VirusTotal, Pappers), les documentations des bibliothèques Python employées, les RFC des protocoles réseau concernés, ainsi que des ressources de référence en matière d'OSINT et de pentest (OWASP, OSINT Framework).

3. Objectifs de la mission

La mission m'a été assignée avec les objectifs suivants :

- Développer un outil en ligne de commande capable d'analyser un ou plusieurs domaines simultanément ;
- Intégrer plusieurs sources de données (DNS, WHOIS, VirusTotal, Hunter.io, scraping web, Pappers) pour produire un profil complet de l'exposition cyber d'un domaine ;
- Implémenter un scan des ports ouverts via Nmap pour identifier les services exposés ;
- Générer des rapports exportables au format PDF et JSON, exploitables directement par les consultants de Cyberses lors de missions d'audit ;
- Développer une interface web Flask permettant de lancer des analyses et de consulter les résultats sans passer par la ligne de commande.

4. Enjeux pour l'entreprise

Dans le cadre de ses missions d'audit et de conseil, Cyberses réalise régulièrement des phases de reconnaissance sur les domaines et infrastructures de ses clients. Sans outil centralisé, ces phases mobilisaient de nombreuses commandes manuelles disparates (dig, nmap, whois, requêtes API individuelles...), ce qui engendrait plusieurs problèmes opérationnels majeurs :

- Perte de temps significative lors de chaque nouvelle mission d'audit, les consultants devant multiplier les outils et consolider manuellement les résultats ;
- Risque d'oubli ou d'incohérence dans la collecte des données selon l'analyste réalisant la reconnaissance ;
- Absence de traçabilité et d'historique des analyses réalisées sur un domaine ;
- Difficulté à produire un rapport synthétique et professionnel directement exploitable par le client.

CyberDiag répond directement à ces enjeux en unifiant l'ensemble de la chaîne de reconnaissance au sein d'un seul outil, reproductible, documenté et livrant un rapport structuré. L'enjeu stratégique est double : gagner en efficacité opérationnelle et professionnaliser les livrables fournis aux clients.

B – Problèmes à résoudre, contraintes et limites

1. Problèmes identifiés

Au démarrage de la mission, l'analyse de l'existant a révélé plusieurs problèmes concrets :

- Fragmentation des outils : les analyses reposaient sur une combinaison d'outils en ligne de commande (nmap, dig, whois) et de consultations manuelles d'APIs tierces, sans aucune centralisation ;
- Absence de rapport automatique : la production d'un rapport de reconnaissance nécessitait une rédaction manuelle chronophage ;
- Pas d'historique : chaque analyse était réalisée ex nihilo, sans capitalisation sur les analyses antérieures d'un même domaine.

2. Contraintes de la mission

Contrainte	Description	Impact
Temporelle	6 semaines pour livrer un outil fonctionnel	Priorisation rigoureuse des fonctionnalités
Légale / Éthique	Analyses uniquement sur domaines autorisés	Tests sur environnements de démonstration
Sécurité API	Clés API sensibles à ne pas exposer dans le code	Gestion via fichier .env non versionné
Maintenabilité	Architecture reprise par l'équipe après le stage	Code modulaire + documentation obligatoires

3. Limites identifiées

Certaines limites ont encadré le périmètre de la mission. Les tests ont été réalisés sur des environnements de démonstration et des domaines fictifs ou autorisés, et non sur les infrastructures réelles des clients en production. De plus, les quotas des APIs gratuites (Hunter.io, VirusTotal) ont parfois contraint les phases de tests intensifs, nécessitant une gestion prudente des appels API. La dépendance à Nmap nécessite également des droits d'exécution suffisants sur le système hôte, ce qui impose une configuration préalable en environnement de production.

II. Démarche suivie pour effectuer la mission

A – Processus suivi et méthode retenue

1. Méthode de travail

Face à la diversité des fonctionnalités à développer et à la contrainte de six semaines, j'ai adopté une démarche de développement itérative inspirée des principes Agile. Le travail a été organisé en cycles hebdomadaires, chacun se terminant par une version fonctionnelle enrichie présentée à Romain Cerruti pour validation et retour.

Cette approche a présenté plusieurs avantages : elle m'a permis d'intégrer les retours du tuteur de manière continue plutôt qu'en fin de projet, d'adapter les priorités lorsque des difficultés techniques imprévues surgissaient, et de maintenir à tout moment un outil opérationnel plutôt qu'un projet inachevé.

2. Présentation du processus adopté

Semaine	Phase	Activités principales
Semaine 1	Analyse	Recueil des besoins, étude des outils existants, prise en main de l'environnement, identification des APIs
Semaine 2	Conception	Définition de l'architecture modulaire, choix technologiques, conception du schéma de données
Semaine 3	Développement cœur	Modules DNS, VirusTotal, Hunter.io, Nmap, intégration dans main.py (CLI argparse)
Semaine 4	Développement avancé	Scraper web, module Pappers, fusion des emails, module d'export PDF
Semaine 5	Interface web	Développement Flask, appel subprocess, gestion résultats temps réel
Semaine 6	Finalisation	Tests bout en bout, corrections de bugs, documentation technique, présentation finale

B – Organisation des missions

1. Suivi et communication

Des points de suivi hebdomadaires avec Romain Cerruti ont ponctué l'avancement du projet. Ces réunions, d'une durée de 30 minutes à une heure, permettaient de présenter les fonctionnalités développées durant la semaine, d'obtenir des retours et de valider ou recadrer les orientations pour la semaine suivante. Des échanges informels quotidiens complétaient ce dispositif pour les questions techniques ponctuelles.

2. Ressources mobilisées

La montée en compétences sur les technologies employées a reposé sur plusieurs types de ressources :

- Documentations officielles : Python 3, Flask, python-nmap, python-dotenv, bibliothèque requests ;
- Documentations des APIs tierces : Hunter.io API v2, VirusTotal API v3, Pappers API ;
- Ressources OSINT : guides méthodologiques de reconnaissance passive (OSINT Framework) ;

Rapport de stage

- RFC des protocoles réseau : RFC 1035 (DNS), RFC 792 (ICMP), RFC 793 (TCP) ;
- Partage de connaissances avec Romain Cerruti et l'équipe Cyberses sur les pratiques réelles d'audit.

III. Réalisation de la mission

A – Justification de la solution retenue

1. Comparatif des solutions existantes

Avant de commencer le développement, j'ai étudié les outils de reconnaissance et d'audit réseau disponibles afin d'évaluer la pertinence d'un développement interne :

Outil	Points forts	Limites pour Cyberses
Maltego	Très puissant, graphes de relations	Coûteux en licence, complexe à déployer quotidiennement
Shodan	Excellent pour la reconnaissance d'IP exposées	Payant, sans rapport PDF automatique ni DNS/WHOIS
theHarvester	Collecte d'e-mails efficace, open-source	Sans intégration DNS/VirusTotal ni export PDF
Nmap seul	Scan de ports de référence	Résultats bruts, non consolidés avec les autres données
CyberDiag (interne)	Sur mesure, multi-sources, rapport PDF auto	Développement initial requis - livré en 6 semaines

Aucun outil existant ne répondait simultanément aux trois critères identifiés par Cyberses : gratuité/open-source, consolidation multi-sources et génération de rapport PDF directement exploitable. Le développement d'un outil interne était donc pleinement justifié.

2. Choix technologiques retenus

Technologie	Rôle dans CyberDiag	Justification du choix
Python 3	Langage principal	Richesse bibliothèques réseau/sécu, maîtrise équipe
Flask	Interface web	Micro-framework léger, déploiement simple
python-nmap	Scan de ports	Wrapper Python officiel autour de Nmap
python-dotenv	Gestion sécurisée des clés API	Séparation config / code source
API Hunter.io v2	Collecte d'e-mails professionnels	Base de données e-mails la plus complète du marché
API VirusTotal v3	Réputation domaine + WHOIS	Référence mondiale en analyse de menaces
API Pappers	Données légales entreprises FR	Seule API gratuite complète sur les SIREN français
BeautifulSoup	Scraping web	Bibliothèque HTML parsing simple et robuste
ReportLab / FPDF	Export PDF	Génération PDF sans dépendance externe lourde

B – Étapes essentielles de la réalisation

1. Architecture modulaire de CyberDiag

CyberDiag a été conçu selon une architecture modulaire strictement organisée, où chaque fonctionnalité est isolée dans un module Python indépendant situé dans le dossier `utils/`. Cette organisation facilite la maintenance, les tests unitaires et l'ajout de nouvelles fonctionnalités sans modifier le cœur du programme.

Le point d'entrée unique est `main.py`, qui orchestre l'ensemble des modules via `argparse` pour l'interface CLI et via `Flask` pour l'interface web. Voici la structure du projet :

cyberdiag/	
├─ main.py	# Orchestrateur principal + CLI (argparse)
├─ app.py	# Interface web Flask
├─ .env	# Clés API (non versionné - .gitignore)
├─ rapports/	# Dossier de sortie des rapports PDF et JSON
├─ utils/	
│ ├── dns_tools.py	# Requêtes DNS (A, MX, NS, TXT, CNAME...)
│ ├── hunter.py	# Collecte d'e-mails via API Hunter.io v2
│ ├── osint.py	# OSINT de base (theHarvester)
│ ├── osint_advanced.py	# VirusTotal API v3 (réputation, WHOIS, menaces)
│ ├── scanner.py	# Scan Nmap des ports ouverts
│ ├── scraper.py	# Scraping web (e-mails exposés, pages)
│ ├── exporter.py	# Génération du rapport PDF
│ ├── pappers.py	# Données légales entreprise (API Pappers)
│ └── pretty.py	# Affichage formaté en console (Rich)

Le diagramme ci-dessous illustre les flux de données entre les différents composants de CyberDiag :

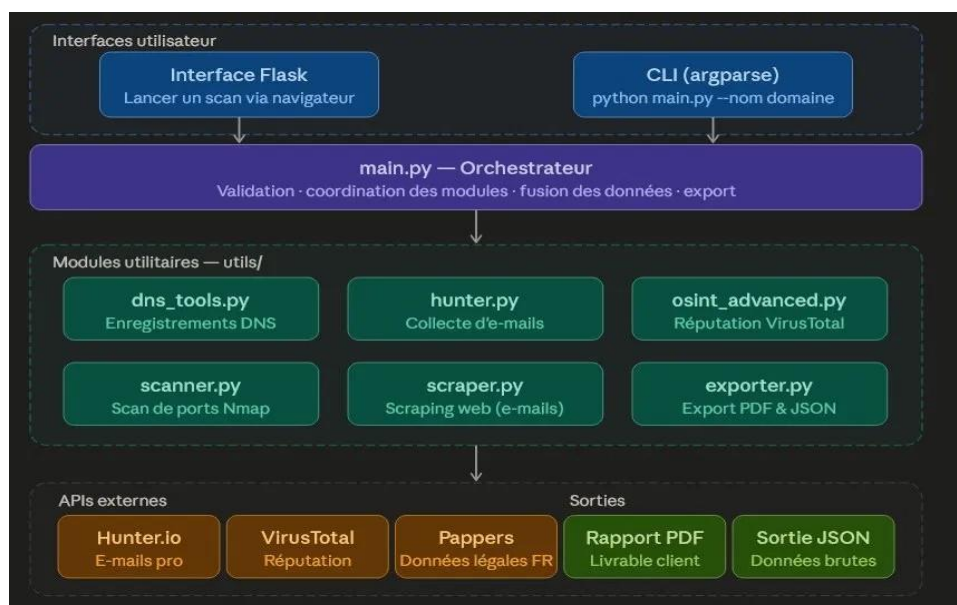


Figure 1 – Architecture modulaire de CyberDiag (interfaces, orchestrateur, modules utils, APIs externes et sorties)

2. Analyse des fonctionnalités développées

Les principales fonctionnalités de CyberDiag sont les suivantes :

Analyse DNS (utils/dns_tools.py)

Le module dns_tools interroge les enregistrements DNS d'un domaine cible (enregistrements A, AAAA, MX, NS, TXT, CNAME). Ces informations permettent d'identifier les serveurs mail, les serveurs de noms autoritaires et les éventuelles configurations SPF/DKIM révélant des services tiers utilisés par l'organisation cible.

Réputation et WHOIS via VirusTotal (utils/osint_advanced.py)

En interrogeant l'API VirusTotal v3, CyberDiag récupère la réputation globale du domaine (score de détection), les données WHOIS (registrar, dates de création et d'expiration, name servers) et l'historique des signalements malveillants. Ces données sont directement intégrées dans le rapport final. Pour le domaine google.com analysé lors des tests, le résultat a confirmé 0 détection malveillante et une réputation saine, ce qui valide le bon fonctionnement du module.

Collecte d'e-mails exposés (utils/hunter.py + utils/scrapper.py)

CyberDiag croise deux sources pour découvrir les adresses e-mail associées au domaine cible. D'une part, l'API Hunter.io v2 est interrogée pour récupérer les e-mails professionnels indexés. D'autre part, le SiteScraper explore les pages du site web (jusqu'à 20 pages configurables) pour y détecter des adresses e-mail directement exposées dans le code HTML. Les résultats des deux sources sont ensuite fusionnés et dédupliqués dans main.py.

Scan de ports Nmap (utils/scanner.py)

Pour chaque adresse IP fournie en argument, le module scanner.py pilote Nmap via la bibliothèque python-nmap pour identifier les ports ouverts et les services associés. Les résultats (port, protocole, état, version du service) sont intégrés dans le rapport global.

Données entreprise via Pappers (utils/pappers.py)

En fournissant un numéro SIREN, CyberDiag interroge l'API Pappers pour enrichir le profil de la cible avec des données légales : dénomination sociale, activité (code NAF), dirigeants, forme juridique et capital social. Cette fonctionnalité est particulièrement utile lors d'audits visant des entreprises françaises.

Export PDF et JSON (utils/exporter.py)

À l'issue de l'analyse, CyberDiag génère automatiquement un rapport PDF structuré et lisible, ainsi qu'un fichier JSON contenant l'intégralité des données brutes. Le PDF est directement remis au client lors des missions d'audit ; le JSON permet une exploitation programmatique des données ou une intégration dans d'autres outils.

Interface web Flask

Une interface web développée avec Flask permet de lancer les analyses depuis un navigateur, de saisir les domaines cibles et de consulter les résultats sans passer par la ligne de commande. Cette interface s'adresse aux consultants moins à l'aise avec le terminal.

3. Difficulté principale rencontrée et résolution

La difficulté technique la plus significative rencontrée durant le stage a été l'intégration de l'interface web Flask avec le script Python principal. Flask est un framework orienté requêtes HTTP synchrones, alors que l'exécution de main.py est une opération longue (plusieurs dizaines de secondes), bloquant totalement le thread Flask le temps de l'analyse.

Dans un premier temps, j'avais tenté d'appeler directement les fonctions Python de main.py depuis les routes Flask. Cette approche bloquait l'interface web, rendant l'application non réactive et impossible à utiliser en production.

La solution retenue, après recherche et échanges avec Romain Cerruti, a consisté à exécuter main.py comme un sous-processus via le module subprocess de Python, en lançant l'analyse en arrière-plan et en récupérant la sortie standard une fois l'exécution terminée. Cette approche a nécessité d'adapter le format de sortie de main.py pour qu'il soit parsable par l'interface Flask, et de gérer correctement les erreurs remontées par le sous-processus.

Leçon retenue : la séparation entre un processus métier long (scan réseau) et une interface web synchrone doit être traitée dès la conception. En production, une migration vers Celery + Redis serait préférable pour un passage à l'échelle.

IV. Évaluation des réalisations et des compétences mobilisées

A – Adéquation du travail

Réaction de Romain Cerruti et de l'équipe

À l'issue du stage, CyberDiag a été présenté à Romain Cerruti lors d'une démonstration finale réalisée sur un domaine de test. L'accueil a été positif. Romain Cerruti a particulièrement souligné la valeur de la fusion multi-sources pour la collecte d'e-mails (croisement Hunter.io + scraping) ainsi que la qualité du rapport PDF généré automatiquement, directement exploitable sans retraitement lors des missions d'audit.

Du point de vue de l'intérêt pour l'entreprise, CyberDiag répond concrètement au besoin initial de centralisation et d'automatisation de la phase de reconnaissance. L'outil permet d'obtenir en quelques minutes un profil complet de l'exposition cyber d'un domaine, ce qui représente un gain de temps estimé à plusieurs heures par mission pour les consultants de Cybersers. L'architecture modulaire retenue facilitera l'ajout de nouvelles sources de données à l'avenir.

Propositions d'amélioration

Amélioration	Description	Priorité
Architecture asynchrone	Passage à asyncio / Celery + Redis pour paralléliser les appels API	Haute
Progression temps réel	Barre de progression et streaming des résultats dans Flask	Haute
Historique BDD	Stockage dans SQLite/PostgreSQL pour comparer dans le temps	Moyenne
Sources OSINT +	Intégration Shodan, Have I Been Pwned, Censys	Moyenne
Conteneurisation	Docker pour simplifier le déploiement et la reproductibilité	Basse

B – Compétences mises en œuvre

Rétrospective des compétences exploitées

Cette mission a mobilisé un large spectre de compétences techniques et transversales, en lien direct avec les référentiels du BTS SIO option SISR.

Compétences techniques développées et renforcées :

- Développement Python avancé : conception modulaire, gestion d'exceptions, manipulation de dictionnaires complexes, fusion et déduplication de données, pilotage de sous-processus ;
- Protocoles réseau : mise en pratique concrète du DNS (types d'enregistrements A, MX, NS, TXT), de TCP/IP (scan de ports) et des protocoles de sécurité associés ;
- Intégration d'APIs REST : consommation des APIs Hunter.io, VirusTotal et Pappers avec gestion de l'authentification par clé, des codes de retour HTTP et des structures JSON ;

- Cybersécurité / OSINT : compréhension et mise en pratique des techniques de reconnaissance passive ;
- Développement web Flask : création de routes, gestion de formulaires, communication interface web / backend Python ;
- Sécurité du code : gestion des secrets via python-dotenv, validation des entrées utilisateur par regex ;
- Versionnement Git : gestion des branches et des commits pour tracer l'évolution du projet.

Compétences transversales :

- Autonomie technique : résolution de problèmes complexes (Flask/subprocess) de manière indépendante ;
- Communication professionnelle : points hebdomadaires structurés avec le tuteur ;
- Rigueur et sécurité : vigilance sur les bonnes pratiques dans le développement d'un outil manipulant des données sensibles.

Montée en compétences : exemple détaillé

La compétence qui a connu la progression la plus marquante est la maîtrise des APIs REST et de l'intégration de sources de données hétérogènes. En début de stage, je savais effectuer des requêtes HTTP basiques avec requests, mais je n'avais jamais travaillé avec des APIs nécessitant une authentification par clé ni géré les quotas d'appels ou la fusion de données multi-sources.

Le processus d'amélioration a consisté à étudier les documentations officielles des trois APIs (Hunter.io, VirusTotal, Pappers), à implémenter progressivement chaque intégration en commençant par des appels simples puis en gérant les cas limites (quota dépassé, domaine inconnu, réponse partielle), et à concevoir la logique de fusion des données e-mail entre Hunter.io et le scraper web.

En fin de stage, je suis désormais capable de concevoir et d'implémenter l'intégration d'une API REST inconnue, de gérer les erreurs et cas limites de manière robuste, et de fusionner des données provenant de sources hétérogènes avec déduplication. Cette compétence est directement transférable à de nombreux contextes professionnels en administration système et en cybersécurité.

Compétences à développer

- Architecture asynchrone : maîtriser asyncio, Celery et la gestion de files de tâches pour développer des applications Python non bloquantes ;
- Conteneurisation Docker : créer et orchestrer des conteneurs pour déployer CyberDiag en environnement de production ;
- Administration Linux avancée : approfondir la gestion des droits, des services systemd et de la supervision ;
- Cryptographie appliquée et protocoles de sécurité : approfondir TLS/SSL et les mécanismes d'authentification.

Conclusion

Ce stage de six semaines chez Cyberses a constitué une expérience professionnelle déterminante pour la suite de mon parcours en cybersécurité et infrastructure réseau. La mission de développement de CyberDiag m'a permis de mobiliser simultanément des compétences en développement Python, en protocoles réseau, en intégration d'APIs et en développement web, dans un contexte professionnel réel et exigeant.

Les conditions de travail au sein de Cyberses ont été très favorables à l'apprentissage. L'encadrement de Romain Cerruti, disponible et pédagogue, m'a permis de progresser efficacement tout en gagnant en autonomie. L'immersion dans un environnement de cybersécurité professionnel m'a confronté à des enjeux concrets que la formation seule ne peut pas restituer : gestion de clés API sensibles, respect des contraintes légales autour des scans réseau, rigueur dans l'architecture logicielle pour garantir la maintenabilité.

L'apport de CyberDiag pour Cyberses est tangible : l'outil est opérationnel, documenté et prêt à être enrichi par l'équipe technique. Il représente une réponse concrète au besoin d'automatisation et de standardisation de la phase de reconnaissance lors des missions d'audit.

Cette expérience a confirmé et renforcé mon intérêt pour la cybersécurité offensive, l'OSINT et le développement d'outils de sécurité. Pour la suite de mon parcours, je souhaiterais poursuivre vers un stage ou une alternance dans un contexte similaire – pentest ou audit de sécurité – afin de compléter mes compétences techniques notamment sur les phases d'exploitation et de remédiation, dans la continuité de ce que j'ai commencé à construire chez Cyberses.

Glossaire

Terme	Définition
API (Application Programming Interface)	Interface permettant à deux applications de communiquer entre elles via des requêtes HTTP standardisées.
ARP (Address Resolution Protocol)	Protocole permettant d'associer une adresse IP à une adresse MAC sur un réseau local.
CIDR	Notation permettant de définir une plage d'adresses IP (ex. 192.168.1.0/24).
CLI (Command Line Interface)	Interface en ligne de commande permettant d'interagir avec un programme via un terminal.
DNS (Domain Name System)	Système de résolution associant des noms de domaine à des adresses IP.
Flask	Micro-framework web Python pour développer des applications web légères.
ICMP	Protocole réseau utilisé pour les diagnostics réseau (ex. commande ping).
Nmap (Network Mapper)	Outil open-source de scan réseau pour découvrir des hôtes et services exposés.
OSINT	Open Source INTelligence : collecte et analyse d'informations à partir de sources publiquement accessibles.
Pappers	Service en ligne donnant accès aux données légales des entreprises françaises via leur SIREN.
Pentest	Test d'intrusion : méthode d'évaluation de la sécurité d'un système par simulation d'une attaque contrôlée.
RFC (Request for Comments)	Documents publiés par l'IETF définissant les standards des protocoles Internet.
SIREN	Numéro d'identification unique des entreprises françaises attribué par l'INSEE.
subprocess	Module Python permettant de lancer et piloter des processus système depuis un programme Python.
VirusTotal	Plateforme permettant d'analyser des fichiers, URL et domaines via de multiples moteurs antivirus.
WHOIS	Protocole permettant d'obtenir des informations sur l'enregistrement d'un nom de domaine.

Bibliographie et webographie

Documentation technique

- Documentation officielle Python 3 - <https://docs.python.org/fr/3/>
- Documentation Flask - <https://flask.palletsprojects.com/>
- Documentation python-nmap - <https://xael.org/pages/python-nmap-fr.html>
- Documentation python-dotenv - <https://saurabh-kumar.com/python-dotenv/>
- Documentation BeautifulSoup4 - <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

APIs utilisées

- Hunter.io API v2 - <https://hunter.io/api-documentation/v2>
- VirusTotal API v3 - <https://developers.virustotal.com/reference/overview>
- Pappers API - <https://www.pappers.fr/api/documentation>

Références réseau et sécurité

- RFC 1035 - Domain Names Implementation and Specification - <https://www.rfc-editor.org/rfc/rfc1035>
- RFC 792 - Internet Control Message Protocol (ICMP) - <https://www.rfc-editor.org/rfc/rfc792>
- OSINT Framework - <https://osintframework.com/>
- Site officiel Nmap - <https://nmap.org/>
- OWASP Testing Guide - <https://owasp.org/www-project-web-security-testing-guide/>
- theHarvester (GitHub) - <https://github.com/laramies/theHarvester>

Annexes

Annexe 1 – Extrait du fichier main.py (orchestrateur CyberDiag)

Le fichier main.py constitue le cœur de CyberDiag. Il assure la validation des entrées, la coordination séquentielle de tous les modules utilitaires, la fusion des données collectées et le déclenchement des exports. Voici un extrait représentatif commenté :

```

1 import os
2 import re
3 from urllib.parse import urlparse
4 from dotenv import load_dotenv
5
6 from utils.dns_tools import dns_lookup
7 from utils.hunter import hunter_search, enrich_emails
8 from utils.osint import osint_harvester
9 from utils.scanner import nmap_scan
10 from utils.osint_advanced import VirusTotalClient, OSINTClient
11 from utils.scrapers import SiteScraper
12 from utils.exporter import export_pdf
13 from utils.parsers import fetch_pappers_data
14 from utils.pretty import afficher_entreprise, afficher_dirigeants
15
16 # --- Validation d'entrée ---
17 def validate_domain(domain: str) -> bool:
18     result = re.match(r"^(?:[a-zA-Z0-9-]{1,})\.[a-zA-Z]{2,}$", domain) is not None
19     import sys
20     sys.stderr.write(f"DEBUG validate_domain: '{domain}' -> {result}\n")
21     return result
22
23 def validate_ip(ip: str) -> bool:
24     return re.match(r"^(?:\d{1,3}\.){3}\d{1,3}$", ip) is not None
25
26 # --- Normalisation domaine ---
27 def normalize_domain(domain_input: str) -> str:
28     try:
29         if "://" not in domain_input:
30             domain_input = "https://" + domain_input
31         parsed = urlparse(domain_input)
32         domain = parsed.netloc or parsed.path
33         domain = domain.lower()
34         if domain.startswith("www."):
35             domain = domain[4:]
36     except:
37         pass
38     return domain
  
```

Figure 2 – Extrait de main.py dans VS Code : imports des modules utils/, validation de domaine et normalisation

Détail des fonctions clés visibles dans l'extrait ci-dessus :

Fonction	Rôle	Lignes
validate_domain(domain)	Vérifie via regex que la chaîne est un nom de domaine valide (format RFC)	20-25
validate_ip(ip)	Vérifie que la chaîne est une adresse IPv4 valide	27-28
normalize_domain(domain_input)	Supprime le préfixe www. et normalise la casse	31-39

La logique d'orchestration principale de main.py enchaîne les appels dans l'ordre suivant :

1. Validation et normalisation du domaine en entrée
2. Appel dns_lookup() → récupération enregistrements A, MX, NS, TXT, CNAME
3. Appel VirusTotalClient() → réputation + WHOIS
4. Appel hunter_search() → e-mails professionnels via Hunter.io
5. Appel SiteScraper() → e-mails exposés sur le site web
6. Fusion et déduplication des e-mails des deux sources
7. Appel nmap_scan() → ports ouverts sur les IPs fournies
8. Appel fetch_pappers_data() → données légales via SIREN (optionnel)
9. Appel export_pdf() → génération du rapport PDF final

10. Sérialisation JSON → sauvegarde du fichier diag_SIREN_date.json

```
# Exemple d'appel CLI :  
python main.py --nom google.com --siren 732075312 --ips 8.8.8.8  
  
# Exemple de sortie console :  
[DNS]      A: 142.250.74.46 | MX: aspmx.l.google.com | NS: ns1.google.com  
[VT]       Réputation: saine | Détections: 0 | Registrar: MarkMonitor  
[HUNTER]   3 e-mails trouvés | scraper: 0 e-mails supplémentaires  
[NMAP]     8.8.8.8 | Ports ouverts: 53/tcp (domain), 443/tcp (https)  
[PDF]      Rapport généré : rapports/diag_732075312_20260512_143256.pdf  
[JSON]     Données brutes : rapports/diag_732075312_20260512_143256.json
```

Annexe 2 – Exemple de rapport PDF généré par CyberDiag

Le rapport PDF ci-dessous a été généré automatiquement par CyberDiag lors d'une analyse test sur le domaine google.com avec le SIREN fictif « cyber ses ». Il illustre la structure du livrable client produit à l'issue de chaque analyse.

Structure du rapport PDF généré : 1. Informations SIREN | 2. Résumé exécutif | 3. WHOIS & Domaine | 4. Certificat SSL/TLS | 5. Headers HTTP | 6. Scraping web | 7. Emails détectés | 8. theHarvester | 9. DNS Records | 10. VirusTotal | 11. Scans IP | 12. Ports détectés

Résultats clés de l'analyse test (google.com) extraits du rapport PDF :

Section	Résultat
Domaine analysé	google.com
Risque global	Faible - 0 détection malveillante
Registrar WHOIS	MarkMonitor, Inc. (depuis le 15/09/1997)
Expiration domaine	14/09/2028
Certificat SSL/TLS	Émetteur : Google Trust Services Valide jusqu'au 13/07/2026
Header Server	gws (Google Web Server)
X-Frame-Options	SAMEORIGIN (protection contre le clickjacking)
Emails détectés	0 (aucun e-mail exposé publiquement)
DNS – Nameservers	NS1.GOOGLE.COM, NS2.GOOGLE.COM, NS3.GOOGLE.COM, NS4.GOOGLE.COM
Ports ouverts	Aucun port détecté (IP de test : 203.0.113.5)

Note : L'absence de résultats DNS (enregistrements A, MX vides) et la non-détection des ports sur 203.0.113.5 s'expliquent par les restrictions réseau de l'environnement de test utilisé pour la démo. En conditions réelles d'audit sur un domaine client autorisé, ces champs sont renseignés.

Annexe 3 – Schéma de l'architecture modulaire de CyberDiag

Le diagramme suivant représente l'architecture complète de CyberDiag, depuis les interfaces utilisateur jusqu'aux sorties, en passant par les modules utilitaires et les APIs externes intégrées.

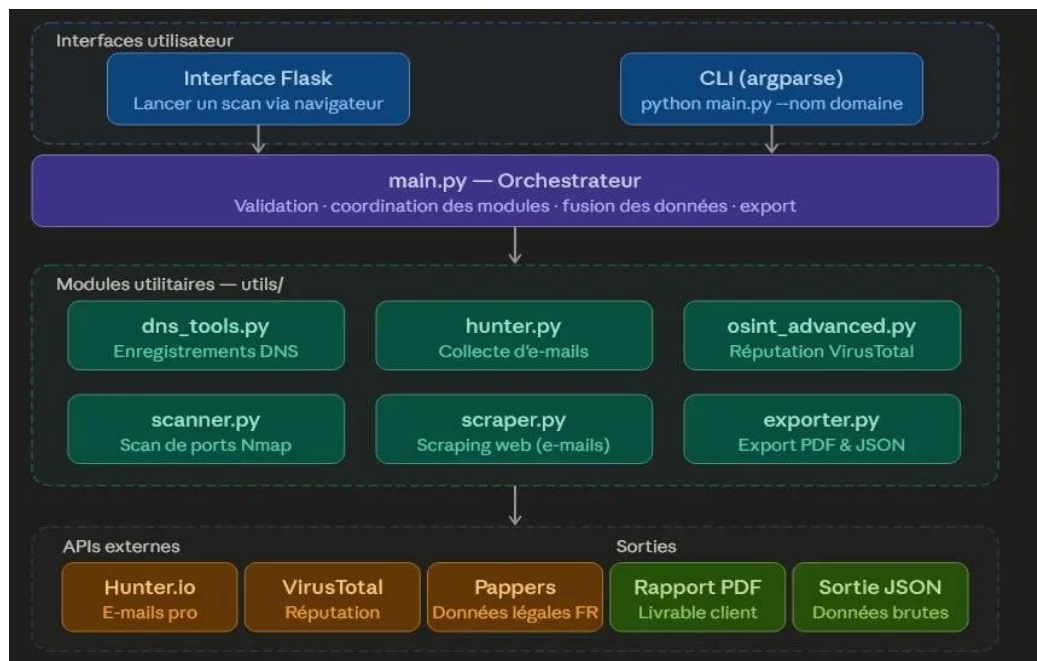


Figure 3 – Diagramme d'architecture CyberDiag : Interface Flask / CLI → main.py (orchestrateur) → modules utils/ → APIs externes (Hunter.io, VirusTotal, Pappers) → sorties (Rapport PDF, JSON)

Lecture du diagramme :

- Les deux interfaces utilisateur (Interface Flask en navigateur et CLI via argparse) délèguent au même orchestrateur main.py ;
- main.py appelle séquentiellement chaque module utils/ selon les arguments fournis ;
- Trois APIs externes sont sollicitées : Hunter.io (e-mails pro), VirusTotal (réputation), Pappers (données légales FR) ;
- Deux types de sorties sont produits : un Rapport PDF structuré (livrable client) et un fichier JSON brut (exploitation programmatique).

Cette architecture modulaire présente plusieurs avantages opérationnels : chaque module peut être testé, mis à jour ou remplacé indépendamment ; un nouveau module peut être ajouté sans modifier main.py au-delà d'un import supplémentaire ; la séparation interface / logique métier / APIs facilite la maintenance sur le long terme.

Annexe 4 – Capture d'écran de l'interface web Flask

L'interface web Flask de CyberDiag permet aux consultants Cyberses de lancer une analyse cybersécurité directement depuis leur navigateur, sans avoir à maîtriser l'interface en ligne de commande.



The screenshot displays the CyberDiag web interface. At the top, a dark blue header contains the logo and the text 'CyberDiag - Plateforme d'Analyse de Securite' with the subtitle 'Analysez vos domaines et collectez des renseignements de securite'. Below this, the 'Nouveau Scan' section features three input fields: 'Nom de l'entreprise' (with an example 'Google, Apple, Microsoft'), 'SIREN (optionnel)' (with an example '732075312'), and 'IP(s) a scanner (optionnel, separees par des espaces)' (with an example '8.8.8.8 1.1.1.1'). A 'Lancer le Scan' button is positioned below these fields. At the bottom, the 'Historique des Scans' section is visible, including a 'Regrouper par:' dropdown menu set to 'Rapport'.

Figure 4 – Interface web Flask de CyberDiag : formulaire de lancement d'un scan (nom entreprise, SIREN optionnel, IPs optionnelles) et historique des scans

Description de l'interface :

- Section « Nouveau Scan » : permet de saisir le nom de l'entreprise cible, un numéro SIREN optionnel (pour enrichissement via Pappers) et une ou plusieurs adresses IP optionnelles à scanner via Nmap ;
- Bouton « Lancer le Scan » : déclenche l'exécution de main.py via un sous-processus Python, sans bloquer l'interface ;
- Section « Historique des Scans » : liste les analyses précédemment réalisées avec possibilité de télécharger le rapport PDF ou le fichier JSON correspondant ;
- Regroupement par rapport : un sélecteur permet de filtrer l'historique par type de rendu.

Contrainte technique résolue : Flask étant synchrone, l'appel à main.py (processus long) a été implémenté via subprocess.Popen() pour éviter le blocage du thread web. Cette architecture sera améliorée via Celery + Redis dans la version suivante.

Annexe 5 – Exemple de fichier de sortie JSON

Le fichier JSON ci-dessous est la sortie brute générée par CyberDiag lors de l'analyse test sur google.com. Il contient l'intégralité des données collectées par tous les modules, structurées par domaine analysé. Ce format permet une exploitation programmatique des résultats ou une intégration dans d'autres outils d'audit.

```
{
  "google.com": {
    "pappers": {},
    "virustotal": {
      "stats": {},
      "results": {},
      "reputation": null,
      "whois_registrar": "MarkMonitor, Inc.",
      "whois_creation_date": ["1997-09-15 04:00:00+00:00"],
      "whois_expiration_date": ["2028-09-14 04:00:00+00:00"],
      "whois_name_servers": ["NS1.GOOGLE.COM", "NS2.GOOGLE.COM",
                            "NS3.GOOGLE.COM", "NS4.GOOGLE.COM"],
      "whois_status": [
        "clientDeleteProhibited",
        "clientTransferProhibited",
        "serverDeleteProhibited"
      ]
    },
    "emails": [],
    "dns": {
      "A": [], "AAAA": [], "MX": [],
      "NS": [], "TXT": [], "CNAME": []
    },
    "ips": {},
    "osint": {
      "error": "ModuleNotFoundError: No module named 'aiohttp_socks'"
    },
    "scraping": {
      "emails": [], "phones": [], "addresses": [],
      "names": ["Accessibility Discovery Centre", "Accessible Places",
                "Guided Frame", "Image Q&A", "Live Caption",
                "Live Transcribe", "Pixel Call Assist",
                "Project Gameface", "Project Relate"],
      "socials": []
    }
  }
}
```

Points d'attention sur ce fichier JSON de test :

- Les enregistrements DNS (A, MX, NS) sont vides car les requêtes DNS ont été bloquées dans l'environnement de test utilisé. En production sur un domaine client, ces champs contiennent les enregistrements réels ;
- L'erreur theHarvester (ModuleNotFoundError: No module named 'aiohttp_socks') indique un problème de dépendance dans l'environnement de test – le module aiohttp_socks doit être installé via pip ;
- La section « scraping.names » contient des noms extraits du site google.com (produits Google accessibilité), ce qui valide le bon fonctionnement du module SiteScraper ;
- Le champ virustotal.reputation à null indique que la clé API VirusTotal utilisée n'a pas retourné de score de réputation - probablement un quota dépassé ou une clé de test limitée ;
- La structure JSON est conçue pour être extensible : l'ajout d'un nouveau module (ex. Shodan) nécessite simplement d'ajouter une nouvelle clé à ce dictionnaire.

Utilisation du JSON en production : ce fichier peut être ingéré dans un SIEM (Security Information and Event Management), un dashboard d'audit ou un script Python de post-traitement pour générer des alertes automatiques sur la base des scores de réputation VirusTotal ou des ports détectés par Nmap.

— Fin du rapport de stage —

EPSI Montpellier - BTS SIO SISR - 2025/2026